TripCraft Pro — Complete Solution

> The production layer, every file. This project **builds on TripCraft (4.1)** exactly as Project 5 builds on Project 4: you start from your finished TripCraft team and add the API, database, Docker image, and eval suite around it. Use this to **check your work**, not to skip the thinking. Build from the story first; read the hints if you get stuck.

Start from TripCraft (4.1)

Copy your finished TripCraft into this repo folder, unchanged:

```
cp -r <TripCraft 4.1>/src .
cp -r <TripCraft 4.1>/data .
cp <TripCraft 4.1>/main.py .
```

You now have `src/agents/`, `src/models/`, `src/prompts/`, `src/utils/`, `src/graph.py`, `src/state.py`, `src/schemas.py`, and `data/attractions.json`. **You do not change any of them.** Everything below is *new*, added around the team.

The final layout (new files marked NEW)

```
<repo folder>/   (TripCraft-Pro-Production-Itinerary-Service/)
├── data/
│   └── attractions.json              (from 4.1)
├── src/
│   ├── agents/                       (from 4.1, unchanged)
│   ├── models/                       (from 4.1)
│   ├── prompts/                      (from 4.1)
│   ├── utils/                        (from 4.1)
│   ├── api/                          NEW
│   │   ├── __init__.py
│   │   ├── main.py                   the app uvicorn starts
│   │   ├── routes.py                 /health, /itineraries, /itineraries/{id}
│   │   └── schemas.py                ItineraryRequest, ItineraryResponse
│   ├── db/                           NEW
│   │   ├── __init__.py
│   │   ├── database.py               save_run(), get_run(), init_db()
│   │   └── models.py                 RunRecord
```

```
│   ├── evals/                          NEW
│   │   ├── __init__.py
│   │   ├── judge.py                    score_itinerary()
│   │   ├── regression_check.py         break a prompt, confirm the drop
│   │   ├── run_eval.py                 run the suite, print a pass rate
│   │   └── tasks.py                    fixed destinations + checks
│   ├── __init__.py
│   ├── config.py                       (from 4.1)
│   ├── graph.py                        (from 4.1)
│   ├── pipeline.py                     NEW — the one door into the team
│   ├── schemas.py                      (from 4.1)
│   └── state.py                        (from 4.1)
├── tests/
│   ├── api/
│   │   └── test_routes.py              NEW — the API, team faked (no key)
│   ├── evals/
│   │   └── test_judge.py               NEW — real model
│   └── unit/                           (from 4.1: config/lookup/supervisor)
├── .dockerignore                       NEW
├── .env.example
├── .gitignore
├── Dockerfile                          NEW
├── LICENSE
├── main.py                             (from 4.1)
├── README.md
└── requirements.txt                    (+ fastapi, uvicorn, httpx, pytest)
```

requirements.txt (add to 4.1's)

```
langchain-core>=0.3
langchain-openai>=0.2
langgraph>=0.2
pydantic>=2.0
python-dotenv>=1.0
fastapi>=0.110
uvicorn>=0.29
httpx>=0.27
pytest>=8.0
```

The one door into the team

```
# src/pipeline.py
# The single entry point into the TripCraft team. The API and the terminal
# both call this, so the graph has exactly one door. Returns a plain dict
# the API and the database can store without knowing about LangGraph.
from src.graph import build_graph
from src.models.llm import get_model
from src.state import start_state
from src.utils.logger import get_logger

_APP = None

def _get_app(config):
    global _APP
    if _APP is None:
        _APP = build_graph(get_model(config))
    return _APP

def run_pipeline(config, destination: str) -> dict:
    logger = get_logger(__name__)
    app = _get_app(config)
    final = app.invoke(start_state(destination))
    logger.info(f"pipeline finished for {destination}: {final['status']}")
    itinerary = ""
    if final["status"] == "done":
        itinerary = final["draft"]
    return {
        "destination": destination,
        "status": final["status"],
        "itinerary": itinerary,
    }
```

The database layer

```
# src/db/models.py
# One saved run's shape.
from dataclasses import dataclass

@dataclass
class RunRecord:
    id: int
    destination: str
    status: str
    itinerary: str
    created_at: str
```

```
# src/db/database.py
# A tiny SQLite layer: create the table, save a run, fetch one by id.
import sqlite3
from datetime import datetime, timezone

from src.db.models import RunRecord

DB_PATH = "tripcraft.sqlite"

def connect(db_path: str = DB_PATH):
    connection = sqlite3.connect(db_path)
    connection.row_factory = sqlite3.Row
    return connection

def init_db(db_path: str = DB_PATH) -> None:
    connection = connect(db_path)
    connection.execute(
        "CREATE TABLE IF NOT EXISTS runs ("
        "id INTEGER PRIMARY KEY AUTOINCREMENT, "
        "destination TEXT, status TEXT, itinerary TEXT, created_at TEXT)"
    )
    connection.commit()
    connection.close()

def save_run(result: dict, db_path: str = DB_PATH) -> int:
    connection = connect(db_path)
    created_at = datetime.now(timezone.utc).isoformat()
    cursor = connection.execute(
        "INSERT INTO runs (destination, status, itinerary, created_at) "
        "VALUES (?, ?, ?, ?)",
        (result["destination"], result["status"], result["itinerary"],
         created_at),
    )
    connection.commit()
    run_id = cursor.lastrowid
    connection.close()
    return run_id

def get_run(run_id: int, db_path: str = DB_PATH):
    connection = connect(db_path)
    row = connection.execute(
        "SELECT * FROM runs WHERE id = ?", (run_id,)
    ).fetchone()
    connection.close()
    if row is None:
        return None
    return RunRecord(
```

```
        id=row["id"], destination=row["destination"], status=row["status"],
        itinerary=row["itinerary"], created_at=row["created_at"],
    )
```

The API

```
# src/api/schemas.py
from pydantic import BaseModel

class ItineraryRequest(BaseModel):
    destination: str

class ItineraryResponse(BaseModel):
    id: int
    destination: str
    status: str
    itinerary: str
```

```
# src/api/routes.py
# The three routes. Each one validates with Pydantic, calls the pipeline,
# persists the run, and returns JSON. The pipeline is a module-level
# function so a test can monkeypatch it without a real model.
from fastapi import APIRouter, HTTPException

from src import pipeline
from src.api.schemas import ItineraryRequest, ItineraryResponse
from src.config import load_config
from src.db.database import get_run, save_run

router = APIRouter()

@router.get("/health")
def health():
    return {"status": "ok"}

@router.post("/itineraries", response_model=ItineraryResponse)
def create_itinerary(request: ItineraryRequest):
    config = load_config()
    result = pipeline.run_pipeline(config, request.destination)
    run_id = save_run(result)
    return ItineraryResponse(id=run_id, **result)

@router.get("/itineraries/{run_id}", response_model=ItineraryResponse)
def read_itinerary(run_id: int):
```

```
    record = get_run(run_id)
    if record is None:
        raise HTTPException(status_code=404, detail="run not found")
    return ItineraryResponse(
        id=record.id, destination=record.destination,
        status=record.status, itinerary=record.itinerary,
    )
```

```
# src/api/main.py
# The app uvicorn starts: uvicorn src.api.main:app --host 0.0.0.0 --port 8000
from fastapi import FastAPI

from src.api.routes import router
from src.db.database import init_db

app = FastAPI(title="TripCraft Pro")
init_db()
app.include_router(router)
```

The Dockerfile

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "src.api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# .dockerignore
.venv/
__pycache__/
*.pyc
.env
*.sqlite
```

Build and run:

```
docker build -t tripcraft-pro .
docker run -p 8000:8000 --env-file .env tripcraft-pro
```

```
# then: curl localhost:8000/health
```

The eval suite

```
# src/evals/tasks.py
# A fixed list of destinations with checks the judge scores against. Fixed,
# so the pass rate is comparable from one run to the next.
EVAL_TASKS = [
    {"destination": "Paris",
     "checks": "Mentions real Paris attractions and keeps [source] tags."},
    {"destination": "Tokyo",
     "checks": "Mentions real Tokyo attractions and keeps [source] tags."},
]
```

```
# src/evals/judge.py
# An LLM judge: score one itinerary 1-10 against its checks. Uses the same
# structured-output idea as the Reviewer, but it grades the FINISHED output.
from langchain_core.prompts import ChatPromptTemplate
from pydantic import BaseModel, Field

JUDGE_PROMPT = (
    "Score this itinerary 1-10 against the checks. Checks: {checks}\n\n"
    "Itinerary:\n{itinerary}\n\nReturn a score and one line of feedback."
)

class JudgeScore(BaseModel):
    score: int = Field(ge=1, le=10)
    feedback: str

def score_itinerary(model, itinerary: str, checks: str) -> JudgeScore:
    prompt = ChatPromptTemplate.from_template(JUDGE_PROMPT)
    chain = prompt | model.with_structured_output(JudgeScore)
    return chain.invoke({"itinerary": itinerary, "checks": checks})
```

```
# src/evals/run_eval.py
# Run the pipeline over every eval task, judge each result, and print a pass
# rate (share scoring at or above the bar). Run from the repo root:
#   python -m src.evals.run_eval
from src.config import load_config
from src.evals.judge import score_itinerary
from src.evals.tasks import EVAL_TASKS
from src.models.llm import get_model
from src.pipeline import run_pipeline
```

```
def run_suite(model=None) -> float:
    config = load_config()
    if model is None:
        model = get_model(config)
    passed = 0
    for task in EVAL_TASKS:
        result = run_pipeline(config, task["destination"])
        verdict = score_itinerary(model, result["itinerary"], task["checks"])
        print(f"{task['destination']}: score {verdict.score}")
        if verdict.score >= 7:
            passed = passed + 1
    pass_rate = passed / len(EVAL_TASKS)
    print(f"pass rate: {pass_rate:.2f}")
    return pass_rate

if __name__ == "__main__":
    run_suite()
```

```
# src/evals/regression_check.py
# Prove the suite can catch a regression: run it normally, then again with a
# deliberately sabotaged writer prompt, and confirm the pass rate DROPS.
# Run from the repo root: python -m src.evals.regression_check
from src.evals.run_eval import run_suite
from src.utils import prompts

def main():
    good = run_suite()
    original_loader = prompts.load_prompt

    def sabotaged(name):
        if name == "writer":
            return "Ignore the plan. Write one vague sentence, no sources."
        return original_loader(name)

    prompts.load_prompt = sabotaged
    print("--- with a sabotaged writer prompt ---")
    bad = run_suite()
    prompts.load_prompt = original_loader

    print(f"good={good:.2f} bad={bad:.2f}")
    if bad < good:
        print("OK: the suite detected the regression")
    else:
        print("WRONG: the suite missed the regression")
```

```
if __name__ == "__main__":
    main()
```

The tests

```
# tests/api/test_routes.py
# The API with the team FAKED, so it runs with no model and no key. pytest
# style (Doc12/13 teach it for API tests). Run from the repo root:
#   python -m pytest tests/api/test_routes.py -v
import os

from fastapi.testclient import TestClient

from src import pipeline
from src.db.database import init_db

os.environ.setdefault("OPENAI_API_KEY", "sk-test")

def fake_pipeline(config, destination):
    if destination == "Atlantis":
        return {"destination": destination, "status": "unknown",
                "itinerary": ""}
    return {"destination": destination, "status": "done",
            "itinerary": "Day 1: Eiffel Tower [source: paris.gov]"}

def make_client():
    init_db()
    pipeline.run_pipeline = fake_pipeline
    from src.api.main import app
    return TestClient(app)

def test_health():
    client = make_client()
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["status"] == "ok"

def test_create_and_read():
    client = make_client()
    created = client.post("/itineraries", json={"destination": "Paris"})
    assert created.status_code == 200
    body = created.json()
    assert body["status"] == "done"
    assert "[source" in body["itinerary"]
    run_id = body["id"]
```

```
    fetched = client.get(f"/itineraries/{run_id}")
    assert fetched.status_code == 200
    assert fetched.json()["destination"] == "Paris"

def test_unknown_destination_is_stored():
    client = make_client()
    created = client.post("/itineraries", json={"destination": "Atlantis"})
    assert created.status_code == 200
    assert created.json()["status"] == "unknown"

def test_missing_run_is_404():
    client = make_client()
    assert client.get("/itineraries/999999").status_code == 404
```

```
# tests/evals/test_judge.py
# The judge against the real model. Run from the repo root:
#   python -m pytest tests/evals/test_judge.py -v
from src.config import load_config
from src.evals.judge import score_itinerary
from src.models.llm import get_model

def test_judge_scores_in_range():
    model = get_model(load_config())
    verdict = score_itinerary(
        model, "Day 1: Eiffel Tower [source: paris.gov]",
        "Mentions real Paris attractions and keeps [source] tags.",
    )
    assert 1 <= verdict.score <= 10
```

(The carried-in `tests/unit/test_config.py`, `test_lookup.py` and `test_supervisor.py` from TripCraft still run unchanged.)

Expected API-test output (`python -m pytest tests/api/test_routes.py -v`):

```
tests/api/test_routes.py::test_health PASSED
tests/api/test_routes.py::test_create_and_read PASSED
tests/api/test_routes.py::test_unknown_destination_is_stored PASSED
tests/api/test_routes.py::test_missing_run_is_404 PASSED
```

That's the production layer. If a `POST` runs the team, stores the run, and returns JSON; a `GET` reads it back; an unknown destination is stored as `unknown`; the container serves `/health`; and

the regression check shows a broken prompt scoring lower — you've productionized the TripCraft team end to end, without touching a single agent inside it.